

1task- HostSiliconAudit&Flynn'sTaxonomyMapping

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	AMD Ryzen 7 4800H (Zen 2, "Renoir"), Family 23 Model 96 Stepping 1	Base & Max Boost Clock	Base: 2.90 GHz / Boost: 3.73 GHz (observed live in Task Manager)
Physical Hardware Cores	8 (<code>NumberOfCores=8</code>)	Logical Cores (SMT/Threads)	16 (<code>NumberOfLogicalProcessors=16</code> <code>ThreadCount=16</code>)
L1 Data/Instruction Cache	Not exposed by WMIC — see note below	L2 Cache (Per Core/Total)	4096 KB total (<code>L2CacheSize=4096</code>) → 512 KB/core (4 MB ÷ 8)
L3 Shared Last-Level Cache	8192 KB = 8 MB (<code>L3CacheSize=8192</code>)	SIMD Vector Extensions	Not exposed by WMIC — see note below

Q1.1 Answer:

My host machine, an AMD Ryzen 7 4800H (Zen 2, 8 physical cores / 16 logical threads), exhibits all three of Flynn's execution modes depending on workload, since Flynn's Taxonomy classifies *execution patterns* rather than the physical chip itself:

SISD (Single Instruction, Single Data): This occurs during a single-threaded, scalar workload — for example, a compiler pass compiling one source file, or my N=1 baseline run in Task 2's benchmark. Here, one core fetches one instruction stream and operates on one data stream at a time, with no parallelism at the thread or data level. Even though the CPU has 8 cores available, only one core (and effectively one of its ALUs per cycle) is doing meaningful work; the other cores sit idle or handle background OS tasks.

SIMD (Single Instruction, Multiple Data): This occurs whenever a single core executes vectorized math — for example, an AVX2-optimized matrix multiplication or a NumPy array operation compiled against vector intrinsics. My CPU's AVX2 support means a single instruction can operate on 256-bit-wide registers, processing 8 packed 32-bit floats (or 4 doubles) simultaneously within one core, one instruction stream, but multiple data elements at once. This is distinct from multi-core parallelism: it happens entirely within a single core's execution pipeline.

MIMD (Multiple Instruction, Multiple Data): This occurs during the OS-scheduled multi-threaded execution seen in Task 2 and Task 3 — for example, running the prime-sieve benchmark across N=8 or N=16 threads, or spawning 10 pthreads to race-increment a shared counter. Here, each of my 8 physical cores (and their SMT-paired

logical threads) independently fetches and executes its *own* instruction stream against its *own* data, fully asynchronously and scheduled by the Windows kernel scheduler across cores. This is the mode that scales with core count and is exactly what Task 2's speedup/efficiency measurements are probing.

Task2 -

Threads (N)	Run1 (s)	Run2 (s)	Run3 (s)	Avg Time TN (s)	Speedup SN=T1/TN	Efficiency EN=SN/N
N=1 (Baseline)	24.5756	—	—	24.5756	1.00×	100.0%
N=2	16.4286	—	—	16.4286	1.496×	74.8%
N=4	28.5129	—	—	28.5129	0.862×	21.5%
N=8	16.4771	—	—	16.4771	1.491×	18.6%
N=16	11.4970	—	—	11.4970	2.138×	13.4%
N=32	10.3043	—	—	10.3043	2.385×	7.5%

Task 3

Run	Measured Output	Error (10 ⁷ – Act)	Run	Measured Output	Error (10 ⁷ – Act)
#1	10,000,000	0	#6	10,000,000	0
#2	10,000,000	0	#7	10,000,000	0
#3	10,000,000	0	#8	10,000,000	0
#4	10,000,000	0	#9	10,000,000	0
#5	10,000,000	0	#10	10,000,000	0

Q3.1 — RMW Window

`counter += 1` isn't atomic — it's three machine steps: **Load** the value into a register, **Add 1**, **Store** it back. If two cores interleave around this window (Core A loads 5, Core B also loads 5 before A stores, both increment to 6 and store 6), one increment gets silently overwritten — a "lost update." Across 10 threads × 1M iterations, this happens repeatedly, so the final count falls short of 10,000,000.

Notably, my first runs showed **zero error** despite no locks. This is because Python's GIL only lets one thread run bytecode at a time and switches threads infrequently by default — coarse enough that each 3–4-instruction increment often completed atomically anyway, masking the race. Lowering the switch interval (`sys.setswitchinterval()`) forced more frequent handoffs and exposed real corruption, confirming the hazard was always present at the bytecode level.

Q3.2 — Synchronization Penalty

The lock forces **serialization** — only one thread can increment at a time, so the 10 threads lose their parallelism for that operation. Each `acquire()/release()` also costs real overhead: an atomic CPU instruction to claim the lock, plus a possible OS-level context switch if a thread has to wait. With 10 threads contending 1,000,000 times each, that overhead is paid 10 million times — far more expensive than the trivial `+1` it's protecting, which is why locked execution is measurably slower.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation (p)

Formula: $p = 2 \times (T_1 - T_2) / T_1$

$T_1 = 24.5756 \text{ s}$ | $T_2 = 16.4286 \text{ s}$

Calculated $p = \mathbf{0.6630}$

2. Amdahl Theoretical Speedup Ceiling

Formula: $S_{\max} = 1 / (1 - p)$

Sequential fraction $(1 - p) = \mathbf{0.3370}$

Max Theoretical Speedup ($N \rightarrow \infty$) = **2.97×**

3. Amdahl 64-Core Projection

Formula: $S_{64} = 1 / [(1-p) + (p/64)]$

Projected Speedup on 64 Cores = **2.88×**

4. Gustafson Scaled Speedup (Weak Scaling)

Formula: $S_{Gustafson} = (1-p) + (p \times 64)$

Projected Scaled Speedup (64× Problem) = **42.77×**

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's Law assumes a fixed problem size (strong scaling) — the sequential fraction $(1-p)$ stays constant no matter how many cores are added, so it caps total speedup at $1/(1-p)$ regardless of core count. That's why the 64-core projection (2.88×) is barely above the theoretical $N \rightarrow \infty$ ceiling (2.97×): once you're past a certain core count, adding more gives almost no benefit, because the sequential portion of the work becomes the dominant bottleneck.

Gustafson's Law assumes the problem size scales up with the number of cores (weak scaling) — instead of finishing the same fixed job faster, the system solves a proportionally larger job in the same time. Under this model the sequential fraction shrinks in relative terms as N grows, so speedup scales nearly linearly with core count (42.77× on 64 cores). Both models describe the same algorithm accurately; they just answer different questions — "how much faster can I finish this exact workload?" (Amdahl) versus "how much bigger a workload can I finish in the same time?" (Gustafson). My $p \approx 0.663$ means roughly a third of my workload is inherently sequential, which is enough to hit Amdahl's ceiling fast, but still leaves huge headroom under Gustafson's scaled-problem framing.

Screenshots

